

HEAP

Priority Queue Study Guide & Decision Framework

Table of Contents

1. Heap Foundations & CBT Structure	Page 2
2. Array Representation & Time Complexities	Page 3
3. Java PriorityQueue API & Checklist	Page 4
4. Pattern 1: Repeated Max/Min Retrieval	Page 5
5. Pattern 2: Top-K Elements	Page 6
6. Pattern 3: Merge K Sorted Lists / Arrays	Page 7
7. Pattern 4: Two Heaps (Median Finder)	Page 8
8. Heap Decision Tree & Invariants Guide	Page 9
9. FAANG Hidden Triggers (Recognition Bible)	Page 10

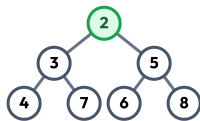
1. HEAP FOUNDATIONS

WHAT IS A HEAP?

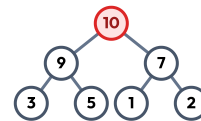
- A Heap is a **Complete Binary Tree** representing a specialized tree-based structure.
- It satisfies the **Heap Property**:
 - **Min Heap**: Parent node values are always $\leq$ child node values. The absolute minimum is at the root.
 - **Max Heap**: Parent node values are always $\geq$ child node values. The absolute maximum is at the root.
- It is **NOT** a sorted data structure. Iterating over the array does not give sorted order.
- Heaps are primarily used to implement **Priority Queues** where fast retrieval of highest/lowest priority element is required.

VISUAL INVARIANT DIAGRAMS

MIN HEAP (Root = Min)



MAX HEAP (Root = Max)



COMPLETE BINARY TREE STRUCTURAL REQUIREMENT

Rule: All tree levels must be completely filled except possibly the bottom level, which must be filled from left to right with no gaps.

✓ COMPLETE

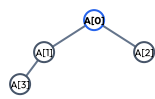


✗ NOT COMPLETE



2. ARRAY REPRESENTATION & COMPLEXITIES

ARRAY INDEX FORMULAS (0-INDEXED)



Parent of node i:	$(i - 1) / 2$
Left Child of node i:	$2 * i + 1$
Right Child of node i:	$2 * i + 2$

OPERATION COMPLEXITY TABLE

Operation	Time Complexity	Space Complexity	Technical Reason
Insert (offer)	$O(\log N)$	$O(1)$	Heapify Up (swapping element up the tree depth)
Remove Min/Max (poll)	$O(\log N)$	$O(1)$	Heapify Down (bubbling root element down to leaf)
Get Optimal (peek)	$O(1)$	$O(1)$	Direct array index 0 root lookup
Build Heap (from N elements)	$O(N)$	$O(1)$	Bottom-Up heapify (internal node heapify is mathematically $O(N)$)

💡 AHA! Bottom-Up Heap construction is $O(N)$. This is significantly more optimal than N sequential insertions which would run in $O(N \log N)$.

3. JAVA PRIORITYQUEUE API

JAVA DECLARATIONS & SYNTAX

1. Min Heap (Default PriorityQueue behavior):

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();  
// Smallest element stays at minHeap.peek()
```

2. Max Heap (3 Implementation Patterns):

• Pattern A: Collections reverse order (Recommended)

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
```

• Pattern B: Integer Comparator class utility

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) → Integer.compare(b, a));
```

• Pattern C: Direct subtraction lambda (Warning: Overflow Risk)

```
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) → b - a);
```

⚠️ WARNING: Avoid subtraction lambda (b - a) when input numbers can be negative. If b is large positive and a is large negative, (b - a) can overflow Integer.MAX_VALUE and flip the comparator sign, causing incorrect sorting bugs! Always use Integer.compare(b, a).

COMMON API METHODS

Method	Behavior	Time
offer(E e)	Inserts element into priority queue.	O(log N)
poll()	Retrieves and removes root element. Returns null if empty.	O(log N)
peek()	Retrieves root element without removal. Returns null if empty.	O(1)
size() / isEmpty()	Returns current size or emptiness check.	O(1)

WHEN TO USE?

Use this pattern when you need to repeatedly query and process the top optimal (smallest or largest) elements from a dynamic stream, modify their values, and insert the results back into the collections.

Common Example Problems: Last Stone Weight (LC 1046), Reduce Array Size to Half (LC 1338).

JAVA CODE TEMPLATE: LAST STONE WEIGHT

```
class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        for (int s : stones) {
            maxHeap.offer(s);
        }

        // Smash 2 largest stones repeatedly
        while (maxHeap.size() > 1) {
            int stone1 = maxHeap.poll(); // Heaviest
            int stone2 = maxHeap.poll(); // 2nd heaviest

            if (stone1 != stone2) {
                maxHeap.offer(stone1 - stone2);
            }
        }

        return maxHeap.isEmpty() ? 0 : maxHeap.peek();
    }
}
```

COMPLEXITY ANALYSIS

Time Complexity:	O(N log N) — Doing N insertions into Heap of size N.
Space Complexity:	O(N) — To store N stones in the priority queue.

WHEN TO USE?

Use this pattern when asked to find the 'K' largest, 'K' smallest, or 'K' most frequent items. Instead of sorting the entire array which takes $O(N \log N)$, we can use a **bound size heap** of size K to solve it in $O(N \log K)$.

Common Example Problems: Kth Largest Element (LC 215), Top K Frequent Elements (LC 347), K Closest Points (LC 973).

JAVA CODE TEMPLATE: KTH LARGEST ELEMENT (USING MIN HEAP)

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        // Min Heap of size K keeps the K largest elements seen so far
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        for (int num : nums) {
            minHeap.offer(num);
            if (minHeap.size() > k) {
                minHeap.poll(); // Remove smallest element out of K+1 elements
            }
        }

        return minHeap.peek(); // Root represents the Kth largest
    }
}
```

AHA SUMMARY & COMPLEXITY

💡 **AHA!** To find K **largest** elements → use a **Min Heap** of size K (discarding smallest). To find K **smallest** elements → use a **Max Heap** of size K (discarding largest).

Time Complexity:	$O(N \log K)$ — Restricting size of heap to K keeps heap operations $O(\log K)$.
Space Complexity:	$O(K)$ — Maximum heap storage.

WHEN TO USE?

Use this when you are given `K` sorted arrays or lists and asked to merge them into a single sorted list. We initialize a Min Heap with the first element of each list, extract the minimum, and pull the next candidate from that list.

Common Example Problems: Merge k Sorted Lists (LC 23), Find K Pairs with Smallest Sums (LC 373).

JAVA CODE TEMPLATE: MERGE K SORTED LISTS

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        // Comparator ensures ListNode elements are sorted by val ascending
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);

        for (ListNode head : lists) {
            if (head != null) pq.offer(head);
        }

        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;

        while (!pq.isEmpty()) {
            ListNode curr = pq.poll();
            tail.next = curr;
            tail = tail.next;

            // Offer the next element from the same list into heap
            if (curr.next != null) {
                pq.offer(curr.next);
            }
        }

        return dummy.next;
    }
}
```

COMPLEXITY ANALYSIS

Time Complexity:	O(N log K) — Where N is total nodes across lists, and K is number of lists.
Space Complexity:	O(K) — Min heap size never exceeds K candidates.

7. PATTERN 4: TWO HEAPS (MEDIAN FINDER)

Partitioning Elements to Query the Running Median

WHEN TO USE?

Use this when asked to maintain a running median from a dynamic stream of values. We divide numbers into two halves: a **Max Heap** stores the smaller half, and a **Min Heap** stores the larger half.

Common Example Problems: Find Median from Data Stream (LC 295), Sliding Window Median (LC 480).

JAVA CODE TEMPLATE: MEDIAN FINDER

```
class MedianFinder {
    private PriorityQueue<Integer> small = new PriorityQueue<>(Collections.reverseOrder()); // Max Heap
    private PriorityQueue<Integer> large = new PriorityQueue<>(); // Min Heap

    public void addNum(int num) {
        small.offer(num);
        large.offer(small.poll()); // Balances max value to min heap

        // Maintain size invariant: small size can exceed large by at most 1
        if (small.size() < large.size()) {
            small.offer(large.poll());
        }
    }

    public double findMedian() {
        if (small.size() > large.size()) {
            return small.peek();
        }
        return (small.peek() + large.peek()) / 2.0;
    }
}
```

COMPLEXITY ANALYSIS

addNum Complexity:	O(log N) — Max two heap offer/poll operations.
findMedian Complexity:	O(1) — Direct peek root lookup.
Space Complexity:	O(N) — Storing N stream elements across the two heaps.

8. HEAP DECISION TREE & CHECKLIST

1. DECISION FLOWCHART (VERTICAL REPRESENTATION)



2. CHECKLIST & AHA MOMENTS

Study Checklist:

- ✓ Always ask: "Which element must come out first?" to determine priority values.
- ✓ Remember to bound the Heap size to K when solving Top-K problems.
- ✓ Avoid integer subtraction overflow bugs by using `Integer.compare(b, a)`.

💡 **AHA! Heap is NOT a sorted array. Only use Heaps when you need $O(1)$ peek access to the root element. If you need arbitrary access or elements sorted, use Trees / Arrays.**

9. FAANG HIDDEN TRIGGERS (RECOGNITION BIBLE)

WHEN TO THINK HEAP? (HIDDEN CLUES)

When the Question Says...	Think Heap Because...
"Top K", "K largest", "K smallest"	You only need to track the top optimal elements (Heap size K).
"K closest points to origin"	Compare elements based on distance metric calculations.
"Kth largest / smallest element"	Retrieves the exact Kth element in optimal $O(N \log K)$ time.
"Most frequent / Top frequent words"	Rank elements using a Heap loaded with HashMap frequencies.
"Running median / dynamic streams"	Requires dividing stream into Max/Min halves (Two Heaps).
"Merge K sorted lists / arrays"	Retrieves the smallest head from lists in $O(\log K)$ repeatedly.

FAANG FAVORITES & TRICKS

Interviewer Tricks to Watch Out:

- They will hide the heap by using priority phrases like "process high importance task first", "retrieve closest distance", or "allocate best resources".
- They will ask for dynamic K values (changing size inputs) → think dynamic heap sizing.
- They ask you to minimize memory constraints when processing billions of inputs → think small size K min heap!

⚠️ PITFALL: Never call `pq.remove(obj)` where `obj` is an arbitrary value in the middle of heap. This is an $O(N)$ linear search operation that completely destroys heap's $O(\log N)$ benefit!